



PROJET SCIENTIFIQUE COLLECTIF

Utilisation des LLM pour la génération de code

22 avril 2025

Coordinateur : Éric GOUBAULT

Tuteur : Joannes VERMOREL

Yiming CHEN - Yuran ZOU - Arthur BUIS

Éric ORDOQUY - Thibaud MONTAGNE



TABLE DES MATIÈRES

I Contexte et objectif du PSC	5
I.1 Introduction : enjeux et intérêts	5
I.2 Objectifs visés	5
I.3 Plan de ce rapport	6
II Etat de l'art	7
II.1 Envision	7
II.2 L'IA générative en programmation	7
II.3 Méthodes d'adaptation des LLM	9
III Modèle Prompt Engineering	11
III.1 Description générale	11
III.2 Implémentation	12
III.3 Système d'évaluation "LLM as a Judge"	12
III.4 Résultats, difficultés	13
IV Modèle "Retrieval-Augmented Generation"	15
IV.1 Description générale	15
IV.2 Implémentation "From Scratch"	15
IV.3 Implémentation avec LangChain	18
IV.3.1 Un plus grand ensemble de benchmarks	19
IV.3.2 Introduction de LangGraph et de FAISS	19
IV.3.3 Notre implémentation	20
IV.3.4 Les différents résultats	22
IV.3.5 Analyse des résultats par catégorie	23
V Pistes d'approfondissement	25
V.1 Filtre de connaissance	25
V.2 Fine-tuning	25
V.3 Un benchmark plus équilibré	25
V.4 Algorithmes de graphe	26
VI Gestion du projet	27
VI.1 Déroulé et organisation du projet	27
VI.2 Enseignements tirés	28
VI.3 Réflexion a posteriori sur notre approche	28
Conclusion	30
Bibliographie	30

ABSTRACT

Ce document est un compte rendu du Projet Scientifique Collectif (PSC) mené tout au long de la deuxième année du cycle ingénieur de l'Ecole polytechnique. Ce PSC consiste en le développement d'un système basé sur un modèle de langage (LLM, pour *Large Language Model*) capable de générer et corriger du code Envision à partir de requêtes formulées en langage naturel. Après nous être familiarisés avec le langage Envision, nous avons exploré plusieurs méthodes et architectures algorithmiques afin de déterminer laquelle fournissait les résultats les plus concluants. Dans ce document, nous commençons par discuter du *Prompt Engineering*, une méthode vouée à fixer une référence d'évaluation pour notre étude, puis nous explorons plusieurs approches de *Retrieval-Augmented Generation* (RAG, ou en français : "génération enrichie par récupération d'information"). Avec cette dernière méthode de recherche contextuelle, nous discutons la pertinence de différentes améliorations que nous avons essayées. Nous analysons tout au long du rapport l'influence de chacun de nos choix sur les résultats obtenus. Nous évoquons enfin les pistes d'approfondissement de notre projet, puis nous concluons sur les apports que nous retirons de l'opportunité qu'est le PSC.

REMERCIEMENTS

Nous souhaitons tout d'abord remercier l'École Polytechnique ainsi que Lokad pour nous avoir offert l'opportunité de travailler sur ce projet. Cela fut pour nous l'occasion d'acquérir de nouvelles compétences techniques et de nous immerger dans un milieu riche et intéressant. Nous sommes extrêmement reconnaissants envers toutes les personnes qui ont contribué à notre projet, et espérons que ce rapport sera à la hauteur de leur investissement.

Pour leur disponibilité, leurs précieux conseils et leur suivi régulier, nous remercions sincèrement Tristan Oualid et Joannes Vermorel sans qui nous aurions eu beaucoup de mal à surmonter les difficultés. Nous remercions également notre coordinateur Éric Goubault, qui a rendu ce projet possible par son encadrement. Nous sommes convaincus que mener ce projet a développé nos compétences techniques et celles de gestion de projet, que nous sommes impatients de valoriser au cours de nos carrières respectives.

I

CONTEXTE ET OBJECTIF DU PSC

I.1 INTRODUCTION : ENJEUX ET INTÉRÊTS

L'utilisation de l'intelligence artificielle (IA) générative comme outil d'aide à la production de contenu est devenue un enjeu majeur de notre décennie. La maîtrise des outils existants, tout comme le développement de solutions sur mesure, offre des gains considérables en productivité et en efficacité dans de nombreux domaines, allant de l'analyse de données au développement logiciel. Ces technologies évoluent rapidement, élargissant sans cesse leur champ d'application.

C'est dans ce contexte que l'entreprise Lokad a exprimé le besoin de faciliter le développement de code dans son propre langage de programmation, Envision. Conçu pour répondre aux besoins spécifiques de l'analyse de données dans le domaine de la supply chain, Envision se distingue par une syntaxe adaptée aux experts métier, tout en s'inspirant de langages comme SQL. Il vise à offrir un compromis entre la simplicité d'Excel et la puissance des outils analytiques de Python, tout en restant accessible à des utilisateurs non développeurs.

Cependant, par définition, les langages spécifiques à un domaine (DSL, pour *Domain Specific Language*) sont spécialisés – ils présentent souvent une syntaxe ou des mots-clés uniques qui sont peu représentés dans les données de préentraînement des LLM. Par conséquent, les LLM amenés à travailler avec ces langages peuvent halluciner et proposer des constructions qui semblent plausibles mais sont invalides.

Même si le code DSL généré est syntaxiquement valide, il peut ne pas faire ce qui est attendu. Garantir la correction sémantique – c'est-à-dire que le code remplisse l'intention de l'utilisateur ou respecte les règles du domaine – est nettement plus difficile. Les LLM ne réalisent pas de raisonnement symbolique par défaut, et peuvent violer des contraintes évidentes pour un expert du domaine. Par exemple, dans un DSL de robotique pour la planification de mouvements, un modèle pourrait produire une séquence d'actions physiquement impossible (comme déplacer un objet sans l'avoir saisi, ou dépasser les limites articulaires), faute de compréhension de la physique sous-jacente aux instructions du DSL.

Notre projet vise donc à explorer la capacité des LLM à comprendre et à générer du code dans ce langage spécifique. Pour cela, nous avons étudié deux approches complémentaires : le Prompt Engineering et la Retrieval-Augmented Generation. Notre ambition est de comparer ces deux stratégies sur des cas concrets, afin d'évaluer leur pertinence et leurs performances dans le cadre d'un langage peu représenté dans les données d'entraînement des LLM.

I.2 OBJECTIFS VISÉS

À la suite de nos échanges avec l'équipe de Lokad, nous avons défini l'objectif principal de ce projet : développer un système basé sur un LLM capable de générer et corriger du code Envision à partir de requêtes formulées en langage naturel. L'idée était de s'appuyer sur un LLM performant d'OpenAI (ou de DeepSeek, dont les modèles sont parus après le début de notre PSC). Pour renforcer leurs capacités, nous avons cherché à leur fournir des instructions optimisées ainsi que des fragments ciblés de documentation.

En parallèle, il nous a fallu une infrastructure d'évaluation permettant de tester rigoureusement les per-

formances du système à travers un ensemble de challenges codés manuellement. Un second LLM a été utilisé comme “juge” pour aider à l’évaluation automatique des réponses proposées et nous permettre d’analyser les résultats obtenus plus efficacement.

Pour nous l’objectif était de se familiariser avec les algorithmes d’IA générative au cours d’un projet concret. Nous souhaitions identifier les limites des pratiques liées à l’application des LLM. Aussi, de manière plus générale, la complexité de la tâche devait nous permettre d’expérimenter la gestion d’un projet en équipe sur le temps long aussi bien en termes de planification que sur la répartition des tâches.

Tous les codes développés dans le cadre de ce projet sont en open-source. Ils sont accessibles aux adresses suivantes : [GitHub.com/VIU-L/LLM-as-judge-for-DSL](https://github.com/VIU-L/LLM-as-judge-for-DSL) et [GitHub.com/SaturnTsen/LLM-DSL-Gen](https://github.com/SaturnTsen/LLM-DSL-Gen).

I.3 PLAN DE CE RAPPORT

Ce rapport s’ouvre sur un état de l’art des modèles de génération de code et du langage Envision. Nous y détaillons ensuite les deux grandes approches explorées : d’abord le Prompt Engineering, puis la Retrieval-Augmented Generation, que nous avons testée dans une version rudimentaire *from scratch*, puis à l’aide du framework LangChain. Chaque méthode fait l’objet d’une analyse critique, nourrie de résultats expérimentaux et d’observations sur leurs limites. Enfin, nous esquissons plusieurs pistes d’amélioration avant de revenir, en guise de conclusion, sur la conduite du projet dans son ensemble.

II

ETAT DE L'ART

Le développement des LLM a bouleversé le domaine du traitement du langage naturel (NLP, pour *Natural Language Processing*), et continue d'offrir de nouvelles applications dans de nombreux domaines. En particulier en génération de code informatique, les LLM se sont rapidement imposés comme l'outil de pointe, du moins pour les langages généralistes et popularisés comme Python. Leur application à des DSL est au contraire un sujet de recherche actuel, et c'est ce qui a motivé notre travail. Dans cette section, nous nous proposons d'établir un panorama des éléments qui ont motivé et nourri notre travail : une présentation du langage Envision puis un suivi de l'évolution des LLM et de leur utilisation pour générer du code informatique.

II.1 ENVISION

Envision est un DSL développé par l'entreprise Lokad, dédié à l'analyse de données dans le domaine de la supply chain. Il est destiné aux experts en supply chain, et permet de manipuler aisément des données concernant les stocks disponibles ou les flux logistiques. Il a été pensé comme un compromis entre la complétude des langages généralistes comme Python et la simplicité de prise en main d'outils comme Excel, afin d'être accessible à l'expert en supply chain sans devoir passer par un ingénieur logiciel. C'est ce qui fait sa principale force. Sa syntaxe est partiellement inspirée de SQL, mais présente de nombreuses différences qui rendent le langage difficile à appréhender pour un LLM standard dont les données d'entraînement ne comprennent pas d'extraits de code ou de documentation Envision. Cette difficulté constitue tout l'enjeu et le défi de notre projet : tester la capacité d'un LLM à apprendre à coder dans un DSL méconnu, au moyen de méthodes d'optimisation d'instructions (Prompt Engineering) et de recherche contextuelle (RAG).

II.2 L'IA GÉNÉRATIVE EN PROGRAMMATION

- HISTOIRE ET NAISSANCE DES LLM

Si les premiers modèles de traitement automatique du langage naturel ont vu le jour pendant les années 1980 avec les premiers modèles probabilistes, c'est en 2017 avec la publication de l'article "Attention is All You Need" [1] que l'architecture de Transformer a été formellement introduite. Initialement conçue pour améliorer la qualité des traductions automatiques, notamment au sein de Google Translate, cette architecture s'est rapidement imposée comme la base des LLM tels qu'on les connaît aujourd'hui. Par la suite, des recherches ont été menées pour identifier quels paramètres devaient être améliorés pour maximiser le rapport entre la performance des modèles et leur coût de développement [2]. Cela, couplé au développement des superordinateurs, a conduit à une accélération du développement des LLM et à l'émergence de modèles de plus en plus puissants et précis, notamment capables d'écrire du code ou de répondre à des questions complexes.

• MODÈLES RÉCENTS ET AVANCÉES

Au cours du déroulement de notre PSC, plusieurs modèles majeurs ont vu le jour, marquant une progression en termes de raisonnement logique et de génération de code. Introduit en mai 2024, GPT-4o [3] s'est d'abord distingué par ses capacités de raisonnement par rapport aux autres modèles à cette époque, et est resté quelques mois le LLM le plus performant pour l'immense majorité des tâches.

Le modèle o1, également développé par OpenAI a vu le jour en septembre 2024. Il a été conçu pour performer dans les tâches nécessitant un raisonnement logique complexe. Par exemple, lors d'un examen de mathématiques de niveau olympique, o1 a atteint un taux de réussite de 83 %, contre seulement 13 % pour GPT-4o [4].

Quelques mois plus tard, en janvier 2025, c'est le modèle DeepSeek R1 qui a été publié. Il a rapidement gagné en popularité grâce à ses performances rivalisant avec o1 en termes de raisonnement logique, et l'arrivée de la chaîne de pensée. Sa popularité s'explique aussi par des coûts d'utilisation par API (pour *Application Programming Interface*) beaucoup plus faibles [5].

Plus récemment, en avril 2025, OpenAI a dévoilé le modèle o3 [6]. Ce modèle paraît très prometteur, avec notamment des performances qui surpassent de loin celles d'o1 sur des benchmarks de référence en matière de résolution de problèmes et de programmation compétitive. Néanmoins, les récentes analyses remarquent aussi une tendance forte à halluciner, et un coût en ressources de calcul plus élevé encore que ses prédécesseurs. Un tel modèle n'aurait donc probablement pas convenu à nos exigences et nos contraintes, même si la question ne s'est pas posée du fait de sa date de parution.

• LA PROGRAMMATION PAR LES LLM

Si la volonté initiale derrière les LLM est de pouvoir générer du texte, notamment pour faciliter les tâches quotidiennes répétitives, l'ambition s'est vite tournée vers la conception d'assistants pour les développeurs en informatique, jusqu'à voir se développer des LLM sachant interpréter et coder dans certains langages de programmation.

Une étude récente [7] classe les requêtes faites aux LLM selon 4 types : les demandes d'apprentissage ou d'exploration, les demandes orientées solution, les demandes de correction d'erreur et les demandes non liées à la solution. Le défi de ce projet scientifique collectif est de permettre au LLM de répondre à des questions de code, ce qui est sensiblement différent, notamment parce que les langages de programmation nécessitent une plus grande rigueur syntaxique. Les avancées technologiques récentes ont permis la réalisation de projets comme Codex [8] ou Starcorder [9], menant à la création, par Fine-tuning ou "from scratch" de LLM capables de générer du code, en l'occurrence sur python concernant ces deux projets. La principale difficulté est l'ensemble des contraintes logiques et syntaxiques qu'imposent les langages de programmation qui ne sont assimilées par ces modèles que de manière partielle ou approximative, en raison de leur formation initiale orientée vers le langage naturel plutôt que vers des structures formelles rigoureuses.

Ce défi est encore plus marqué lorsqu'il s'agit des DSL, qui possèdent une syntaxe particulière et sont souvent peu représentés dans les données d'entraînement des LLM. Pour les adapter efficacement, les chercheurs utilisent des stratégies telles que le fine-tuning, le Prompt Engineering, ainsi que la génération augmentée par récupération (RAG), qui injecte des connaissances externes comme la documentation d'API. D'autres approches consistent à affiner progressivement les générations en s'appuyant sur les résultats d'exécution pour corriger les erreurs ou encore à concevoir des langages spécialisés intermédiaires, dont la structure est plus accessible aux modèles, afin de faire le lien entre des instructions textuelles usuelles et des langages de programmation moins répandus [10].

Il n'existe pas de benchmark universel permettant l'évaluation de la génération de code provenant de DSL. Les chercheurs s'appuient donc sur des jeux de données personnalisés et des métriques spécifiques propres au langage considéré, telles que la validité syntaxique, la correspondance exacte ou le taux de succès à l'exécution. Parmi

les benchmarks courants, on trouve MultiPL-E [11], SMCaFlow [12] et Spider [13] pour l'analyse sémantique et SQL, ou encore NVBench [14] pour la visualisation de données. Ces benchmarks, bien qu'originellement conçus pour des langages usuels, peuvent être adaptés à des DSL en modifiant les jeux de données, les tâches cibles ou les critères d'évaluation, fournissant ainsi une base méthodologique réutilisable pour structurer l'évaluation dans des contextes moins standardisés.

Le Fine-tuning des LLM sur des tâches spécifiques permet d'obtenir des améliorations concrètes en génération de code. Toutefois, pour garantir la fiabilité des résultats, ces modèles sont souvent associés à des approches hybrides, combinant des mécanismes de validation par essais successifs et l'ajout de connaissances externes, telles que la documentation du langage. Depuis 2022, les avancées dans ce domaine ont permis de démontrer que ces solutions sont techniquement réalisables et applicables à une grande variété de DSL, tels que Verilog [15], MPI [16] ou encore YAML [17]. Bon nombre de ces capacités sont progressivement intégrées dans les outils de développement, comme les extensions VS Code pour Azure Bicep ou les templates AWS CloudFormation utilisant Codex. À mesure que les modèles s'améliorent et que davantage de données de domaine sont incorporées, il est raisonnable de s'attendre à ce que les LLM deviennent des assistants standards pour les tâches de programmation spécifiques, augmentant significativement la productivité dans des domaines qui, jusqu'alors, nécessitaient une expertise très pointue des DSL.

• LIMITES DES MODÈLES

Malgré les performances remarquables que nous venons de présenter, les LLM présentent plusieurs limites bien documentées. Leur raisonnement peut être biaisé par leurs données d'entraînement, certaines réponses sont hallucinées, et la rigueur syntaxique nécessaire en programmation n'est pas toujours respectée, même si les performances sur ce point ne cessent de progresser. Des études critiques sur les modèles récents, notamment o1 [18], suggèrent que certaines bonnes performances pourraient être expliquées par une forme de mémorisation plus que par un raisonnement réel. Une limite majeure qui découle de cette étude est le besoin continu de développer de nouveaux modèles entraînés sur des données récentes, car un modèle qui échoue à raisonner sera vite dépassé face à des problèmes nouveaux.

Ces limites sont d'autant plus vraies dans le cas des DSL, où les LLM peuvent transférer à tort des habitudes syntaxiques issues d'autres langages, faute de documentation suffisante. C'est dans ce contexte que s'inscrit notre projet, puisque le langage Envision n'est pas connu des LLM les plus performants.

II.3 MÉTHODES D'ADAPTATION DES LLM

Pour renforcer les capacités ou les connaissances d'un LLM existant sur un domaine ou une tâche spécifique, plusieurs méthodes sont aujourd'hui utilisées. Ce paragraphe vise à explorer les différentes méthodes que nous avons envisagées et justifier notre choix final.

Le terme Prompt Engineering désigne simplement le fait de rechercher un prompt qui va susciter le comportement souhaité chez le LLM. Par exemple, si l'on souhaite utiliser un modèle comme assistant pédagogique, cette approche consiste à concevoir un prompt qui le guidera vers un rôle d'explication clair et structuré. Il s'agit d'une méthode empirique, très simple à mettre en oeuvre. Elle permet d'identifier rapidement ce dont est capable ou non un modèle avant d'y ajouter des méthodes d'adaptation supplémentaires.

Ensuite, la Retrieval-Augmented Generation est une méthode de recherche ciblée de contenu susceptible d'apporter au modèle des éléments de réponse à la requête de l'utilisateur. Pour reprendre l'exemple précédent, la RAG consiste à ajouter au LLM tuteur une base de données de cours. Ainsi, lorsque l'utilisateur spécifiera dans sa requête le nom du cours qu'il souhaite réviser, une recherche ciblée viendra ajouter automatiquement

au prompt reçu par le LLM le cours mentionné s'il se trouve dans la base de données. Cela assure que le LLM reprend les mêmes conventions que celles du cours qu'à suivi l'utilisateur. Cette méthode est assez peu coûteuse à mettre en place, mais comprend deux difficultés principales : premièrement, construire un algorithme de recherche ciblée adapté aux données avec lesquelles on travaille (typiquement, l'algorithme varie lorsqu'on travaille avec une base de données de code et non de langage naturel) ; deuxièmement, structurer la base de données de manière à ce que les morceaux ajoutés aux prompts utilisateurs soient ni trop longs, ni trop courts. Ces points seront développés dans la section IV.

Une troisième méthode d'adaptation de LLM est le *Fine-Tuning* (en français : "recalibrage du modèle"). Contrairement aux deux précédentes, celle-ci est paramétrique : elle consiste à réentraîner le LLM sur un jeu de données de manière à changer ses coefficients, dans le but de lui faire apprendre des connaissances et des raisonnements spécifiques au sujet d'intérêt. Cette méthode est largement plus difficile à mettre en place, puisqu'il faut concevoir un ensemble de questions-réponses qui serviront de données d'entraînement. En termes de coût, rappelons que l'algorithme de rétropropagation qui est à la base de la phase d'apprentissage d'un LLM a une complexité linéaire en le nombre de paramètres du modèle (pour une seule paire question-réponse), et ce nombre s'exprime en centaine de milliards pour les LLM performants : le coût en ressources de calcul et en ressources monétaires est donc élevé. Nous avons choisi de garder l'exploration de cette méthode pour la fin de notre PSC, dans l'éventualité où nous ne pourrions plus aller beaucoup plus loin avec la méthode de RAG.

L'apprentissage par renforcement peut également être utilisé comme méthode paramétrique afin d'ajuster les coefficients du LLM. Le terme d'apprentissage par renforcement désigne un cadre d'apprentissage où un agent, ici le LLM, apprend à prendre des décisions en interagissant avec un environnement, en recevant une récompense ou une punition à chaque action. L'objectif de l'agent est de maximiser la récompense cumulée à long terme. Cette classe de méthodes d'apprentissage comprend plusieurs applications de pointe pour les dernières phases d'apprentissage, mais nous avons décidé de ne pas nous y consacrer car elle constitue un prolongement du Fine-Tuning, dont la réalisation nous paraissait déjà un objectif ambitieux.

III

MODÈLE PROMPT ENGINEERING

III.1 DESCRIPTION GÉNÉRALE

Le premier modèle que nous avons développé fonctionne avec la méthode Prompt Engineering définie précédemment : elle consiste à donner à un LLM existant un prompt lui expliquant :

- son rôle de développeur informatique
- un extrait de documentation Envision qui permet au modèle de comprendre la syntaxe du langage
- le challenge auquel il va devoir répondre

Concrètement, concevoir ce modèle signifie donc écrire le texte expliquant au LLM son rôle et choisir l'extrait de documentation fourni. Pour le texte de rôle, nous avons essayé différents formats, et conformément à ce que nous avons pu lire dans les articles scientifiques [19], ce qui fonctionne le mieux est un prompt clair, structuré et surtout concis. Nous avons convergé vers la version finale ci dessous :

```
docu = read_file(os.path.join("docs", "envision-brief.md"))

coder_personality = "You are a proficient coder in a Domain Specific Language called Envision. \
    Your task is to generate response to the given challenge. \
    Some challenges will ask you to generate Envision code,\
    others will ask you to explain given code or answer questions related to the Envision language. \
    Do not output any intermediate thinking or explanation, only give the final answer.\
    Here is the documentation of Envision:\
    ## Documentation\n" + docu
```

FIGURE 1 – Prompt donné au LLM-codeur

Pour l'extrait de documentation, nous avons rapidement convenu que le meilleur choix est de donner au LLM la documentation résumée du langage Envision, un document d'environ 20 pages intitulé *Envision brief* qui donne la majeure partie des bases syntaxiques du langage. Nous aurions pu donner un extrait de documentation plus gros, car la limite de caractères pour un prompt était encore loin d'être atteinte, par exemple en ajoutant les deux premiers chapitres de la documentation complète. Mais cela aurait donné à notre modèle des compétences déséquilibrées, plus développées pour les notions des deux premiers chapitres et pas des autres. Nous souhaitons éviter de donner ces capacités déséquilibrées à notre premier modèle, puisque notre objectif avec ce premier modèle était de fixer une référence de performance pour les modèles futurs.

Pour résumer, cette méthode est limitée par la faible quantité d'informations sur le langage Envision qu'elle fournit au modèle (un prompt donné à une IA générative est en effet soumis à une limite de *tokens*, *i.e.* de morceaux de mots). Le but de ce modèle est simplement de déterminer les capacités génératives des LLM en l'absence d'information ciblée, ce qui permettra de mesurer la plus value due à l'ajout d'information ciblée avec la méthode RAG.

III.2 IMPLÉMENTATION

Pour l'implémentation, nous avons choisi d'utiliser l'API d'OpenAI, dont les LLM étaient alors les plus performants dont nous disposions. Nous avons également structuré la documentation Envision dans des fichiers markdown, défini des fonctions de lecture de document permettant de traduire ces fichiers en chaînes de caractères, et conçu une pipeline (*i.e.* une fonction de traitement) permettant d'exécuter le modèle. Ensuite, afin de pouvoir l'évaluer, nous avons construit un premier benchmark (*i.e.* données d'évaluation) composé de 15 problèmes exprimés en langue anglaise, tous munis d'une correction détaillée et commentée en langage Envision. Nous y avons volontairement incorporé des problèmes de difficulté variable, et dont les solutions nécessitent un éventail de fonctions suffisamment large et représentatif du langage. Ce benchmark étant de taille assez restreinte, nous avons commencé par observer manuellement les résultats produits par notre premier modèle, mais la perspective de l'élargissement futur du benchmark nous a poussé à implémenter une méthode d'évaluation automatisée, inspirée de la méthode "LLM as a Judge" [20]. Elle consiste à confier l'évaluation des codes fournis par le LLM codeur pour chaque challenge à un LLM dit "juge", pour obtenir un score final sur l'ensemble des challenges.

Nous avons ensuite augmenté cette pipeline d'un passage par un compilateur Envision afin d'éliminer directement les résultats qui ne compilent pas, sans les soumettre au LLM juge. Tout code qui ne compile pas est en effet nécessairement un échec au challenge. Pour tenir compte du caractère aléatoire de la génération de code par un LLM, nous autorisons le LLM codeur à soumettre jusqu'à deux propositions supplémentaires en cas d'échec à la compilation. Finalement, notre pipeline fonctionne selon l'architecture suivante :

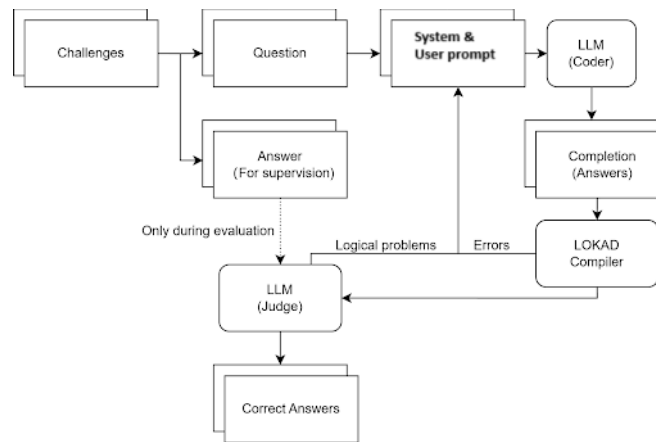


FIGURE 2 – Pipeline de notre modèle, avec son système d'évaluation

III.3 SYSTÈME D'ÉVALUATION "LLM AS A JUDGE"

Cette sous-section vise à détailler le fonctionnement de notre système d'évaluation, et discuter des choix algorithmiques auxquels nous avons été confrontés.

Dans la pipeline présentée ci-dessus, le LLM juge reçoit un prompt contenant ses consignes de comportement, le code soumis par LLM codeur, le challenge auquel il répond ainsi que la correction de ce challenge. Nous avons premièrement observé que le juge gagne en précision lorsqu'il est guidé dans sa correction, donc nous avons agrémenté les corrections des challenges de notes pour s'assurer que le juge vérifie correctement les points

cruciaux du code à évaluer. Ensuite, comme pour le LLM codeur, nous avons affiné le texte de comportement du juge pour qu'il explique en retour son raisonnement, c'est-à-dire pourquoi il a choisi de valider ou d'invalidier le résultat du LLM codeur, point par point, et donner finalement une note binaire de 0 ou 1.

Le choix de demander une notation binaire pour chaque challenge est arbitraire : nous aurions par exemple pu exiger une note entre 0 et 10 pour chaque challenge, et retenir comme note finale la moyenne des notes sur chaque challenge. Nous avons néanmoins choisi de ne pas explorer cette deuxième possibilité, notamment car elle est plus dépendante des erreurs du juge, et elle exige de construire un barème détaillé pour chaque challenge, pour éviter de laisser intervenir le "libre-arbitre" du LLM juge. Cela représenterait une charge de travail colossale en cas d'élargissement du benchmark, et risquerait de produire une notation finale assez peu transparente. En comparaison, notre système d'évaluation binaire a certes moins de nuances, mais il a l'avantage de pouvoir identifier directement les échecs du LLM codeur, et donc de comparer les résultats produits par différents modèles. En outre, ce système binaire permet d'identifier plus simplement les éventuelles erreurs du juge, qui est loin d'être infallible.

Ce système d'évaluation est néanmoins limité. Comme nous l'avons précisé, le juge est amené à faire des erreurs d'évaluation. Pour les plus récurrentes, nous avons réussi à les corriger en ajoutant des notes dans son texte de comportement. Par exemple, nous avons observé que le juge invalidait systématiquement les réponses du codeur lorsque ce dernier utilisait un paramètre d'affichage différent de la correction. Pour résoudre ce problème nous avons précisé le canevas de paramètre d'affichage au juge dans son texte de comportement et nous lui avons ajouté la consigne de ne pas s'y intéresser.

En revanche, certaines erreurs continuent à survenir de manière impromptue sans que nous parvenions à les expliquer. Prenons pour exemple un challenge d'affichage d'une distribution de probabilités, où le juge sanctionne parfois le codeur car il n'a pas donné de nom à la figure affichée. Ce cas indique que l'on ne peut pas se fier aveuglément à la note chiffrée obtenue avec la méthode LLM as a Judge, et qu'une analyse manuelle reste nécessaire afin d'obtenir un score exact. La note obtenue s'interprète donc plutôt comme une mesure accompagnée d'une incertitude, que l'on a fait en sorte de réduire en adaptant les notes de correction. Dans le cas de notre premier benchmark de 15 challenges, cette incertitude approchait ± 1 .

Finalement, implémenter ce système d'évaluation nous a permis d'être plus efficace dans notre évaluation des résultats, notamment lorsqu'il s'agissait de comparer plusieurs modèles ou d'identifier leurs points faibles, même si il peut introduire un léger biais sur les scores chiffrés qui seront présentés dans le présent rapport et dans notre soutenance orale.

III.4 RÉSULTATS, DIFFICULTÉS

Les premières expériences réalisées avec le modèle Prompt Engineering se sont avérées assez satisfaisantes : pour les challenges ne nécessitant pas de fonction non mentionnée dans la documentation résumée ni de mots clés avec une syntaxe trop spécifique, nous avons rapidement obtenu des codes satisfaisants, en utilisant le modèle GPT-3.5 comme LLM codeur.

Un point intéressant de l'analyse de ces premiers résultats est la confrontation de différents LLM. Quelques itérations du programme montrent qu'on obtient un score moyen de 8,5/15 avec le LLM GPT-3.5, où l'indétermination sur le score provient d'un challenge d'explication de code que le codeur n'arrive à résoudre qu'une fois sur deux en moyenne. Paradoxalement, le LLM GPT-4o-mini n'obtient pas un meilleur score : son score moyen est de 7,5, avec des erreurs qui surviennent cette fois sur différents challenges. Pourtant, GPT-4o-mini est un modèle plus puissant que GPT-3.5, qui obtient de meilleurs résultats sur les tâches usuelles. L'explication de ce problème réside dans les limites de notre système d'évaluation : le LLM GPT-4o-mini inclut dans son code des

paramètres facultatifs, et le LLM juge identifie parfois ces paramètres comme une erreur car ils peuvent différer de ceux proposés en correction, sans pour autant être erronés. Le modèle GPT-3.5 propose un code plus simple, sans commentaires ou paramètres additionnels. En d'autres termes, GPT-4o-mini propose un code mieux commenté et plus complet que GPT-3.5, mais moins proche de la correction du challenge, ce qui est sanctionné à tort par le juge. La comparaison à ce stade avec d'autres modèles plus performants comme DeepSeek-R1 aurait été intéressante, mais le modèle a été lancé plusieurs mois après notre implémentation.

Finalement, en observant les performances du modèle Prompt-Engineering, qui reçoit pour seule source d'information sur le langage en vision sa documentation résumée, on remarque que la principale limite n'est pas la compréhension de cette documentation mais le manque d'information qui y est présente. Pour espérer obtenir des modèles plus performants, il faut donc parvenir à fournir plus d'information au LLM-codeur, et de préférence de l'information ciblée, ce qui motive l'implémentation du modèle RAG.

IV

MODÈLE “RETRIEVAL-AUGMENTED GENERATION”

IV.1 DESCRIPTION GÉNÉRALE

Le problème de manque d'information contextuelle a donc motivé une solution par la RAG. Elle consiste, comme expliqué en section II.3, à aller chercher de l'information dans une base de données textuelles pour compléter les requêtes utilisateurs reçues par le LLM, et donc lui donner des éléments de réponse supplémentaires. Par exemple, un des objectifs dans notre cas est de donner au LLM un accès direct aux connaissances les plus utiles pour une requête donnée, afin d'éviter les erreurs de syntaxe par exemple.

La tâche de sélection des informations "pertinentes" à ajouter à une requête n'est pas facile. La pratique générale consiste à créer un embedding (*i.e.* une représentation vectorielle) pour chaque pièce d'information de la base de données, et de sélectionner celles qui ont l'embedding le plus similaire à celui de la requête utilisateur. Un choix courant pour la notion de similarité est la similarité-cosinus, qui calcule le cosinus de l'écart d'angle entre les deux vecteurs et permet donc de sélectionner les vecteurs de la base de données qui ont la direction la plus proche du vecteur requête.

Embedding et Cosine similarity

L'embedding est une représentation vectorielle d'éléments dans un espace à plusieurs dimensions. Son but est de conserver les relations entre des mots mais de manière transcritible à l'ordinateur. Par exemple, dans un espace d'embedding, les mots "chat" et "chien" seront proches l'un de l'autre, tandis que "chat" et "voiture" seront éloignés. Cela permet aux modèles de langage de mieux comprendre le sens des mots et leur contexte.

Par proches, nous entendons que le produit scalaire entre les deux vecteurs est grand. En d'autres termes, si a et b sont deux vecteurs, la cosine similarity est définie par :

$$\text{cosine_similarity}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}} \quad (1)$$

où $\|a\|$ et $\|b\|$ sont les normes des vecteurs a et b . La cosine similarity varie entre -1 et 1, où 1 signifie que les deux vecteurs sont identiques, 0 signifie qu'ils sont orthogonaux (sans relation) et -1 signifie qu'ils sont opposés.

Dans notre cas, la base de données textuelles utilisée est la documentation du langage Envision. Nous avons essayé différentes méthodes pour découper cette documentation en morceaux, et plusieurs méthodes de recherche par similarité différentes : l'objet de cette section est de présenter nos implémentations et de confronter leurs résultats.

IV.2 IMPLÉMENTATION "FROM SCRATCH"

• PREMIÈRE IMPLÉMENTATION

Pour notre premier essai, nous avons implémenté la version la plus générale de la RAG, avec la similarité-cosinus. Cette première implémentation a été l'occasion pour nous de nous confronter au problème de découpage de la documentation. La première version naïve a été de découper la documentation en morceaux de longueur égale. Si cette solution peut fonctionner pour les textes continus, elle n'a pas été pertinente dans notre cas puisqu'elle a divisé certains extraits de code de la documentation en deux morceaux. Pour éviter ce premier écueil, nous avons pris la décision d'effectuer une segmentation par cadres de sous-sections, puis par cadres de sous-sous-sections afin d'obtenir des morceaux d'une longueur moyenne acceptable. Le dernier ajustement que nous avons fait à ce stade est de pénaliser le score des morceaux de trop grande longueur, qui risquaient d'être choisis préférentiellement car ils ont plus de contenu.

L'architecture de la pipeline devient donc celle-ci :

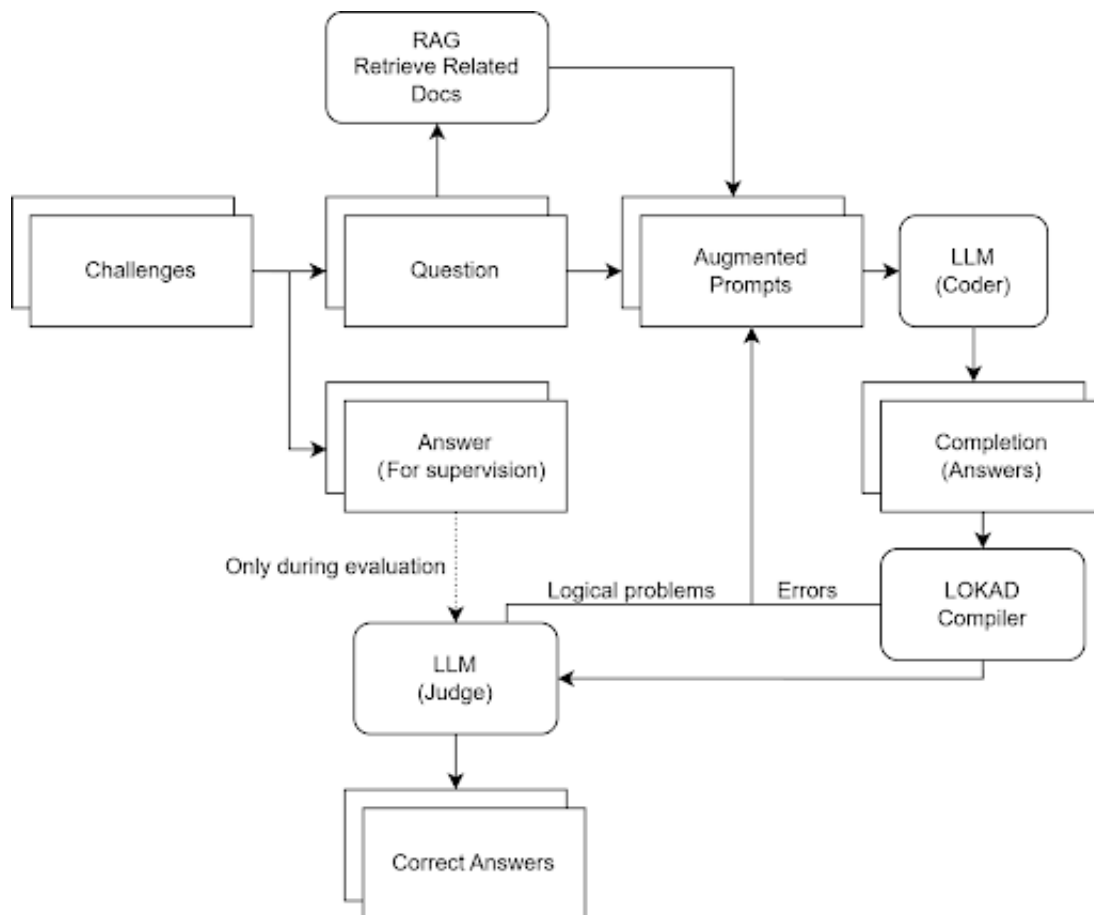


FIGURE 3 – Pipeline de notre modèle, avec le module RAG

Pour créer les embeddings nous avons utilisé l'outil 'sentence-transformers/multi-qa-mpnet-base-dot-v1'; nous choisissons les 5 morceaux de grammaire et les 5 morceaux de documentation écrite qui ont le meilleur score, en tenant compte de la pénalité imposée aux morceaux de grande longueur.

Avec cette première approche, sur le benchmark de 15 questions, nous avons 8-9 réponses compilables et 7-8 réponses jugées correctes, selon l'aléa. Cette méthode n'apporte donc pas d'amélioration de score sur le modèle, elle a au contraire diminué ses performances. Une analyse au cas par cas nous a montré que les morceaux extraits

sont très souvent peu en rapport avec la question. Inclure cette information supplémentaire inutile brouille la compréhension du LLM, le conduisant à halluciner. Ce problème vient notamment de la difficulté à créer un espace d'embeddings pertinent pour des extraits textuels mélangeant langue anglaise et lignes de code, plus particulièrement pour le langage Envision qui est peu utilisé et donc ne fait partie des données d'entraînement d'aucune base de données textuelles.

• AMÉLIORATION : RAG HYBRIDE

Pour mieux cibler les documents pertinents, nous nous sommes proposés d'ajouter une structure qui s'appelle "RAG demander" : nous prenons un deuxième LLM, qui en lisant la question et la liste de bibliographie, propose des catégories grammaticales et des noms de fonctions. Voici l'instruction de comportement qui lui est assignée :

```

1  demander_personality = '''You are a coder's assistant. The coder will be
    coding in a Domain Specific Language called Envision. You are given the
    BASIC DOCUMENTATION of Envision: \n&BASIC DOCUMENTATION''' + docu + \
2  '''END OF BASIC DOCUMENTATION &\n. You are also given the REFERENCES of
    Envision:\n&REFERENCES\n''' + REFERENCES + \
3  '''\nEND OF REFERENCES &\nThe user will give you a CODING TASK as input. You
    shall not try to code yourself; instead, based on the BASIC DOCUMENTATION
    you have seen above, you shall propose
4  1.some grammar chapters listed in the "### Grammer Chapters" section in the
    REFERENCES;
5  2. some functions listed in the "### Aggregators and Functions Available"
    section in the REFERENCES, that a coder will need to know in order to
    complete the CODING TASK. Your response MUST be something from these 2
    sections. Attention that function categories are not grammar chapters.
    Separate your points with a line break. Start each line with a - for
    grammar chapter and + for a name of the aggregator/function. Example: -
    Relational algebra (line break) + dirac (line break) + chineseyear.
6  Do not give any intermediate thinking nor explain why you need that
    information. You should at least propose 5 Aggregator/function usages and
    3 grammar chapters, but you can propose more if you think it is necessary.
    '''

```

Listing 1 – RAG-demander

Ensuite, nous effectuons la recherche seulement sur les catégories proposées. Nous prenons aussi directement les morceaux de documentation similaires aux noms des fonctions proposées. Ainsi, nous espérons une requête plus précise en mobilisant la connaissance générale d'un LLM et le fait que le nom d'une fonction en Envision traduit souvent bien son domaine d'utilité. Si le texte de la question de l'utilisateur ne révèle pas forcément les fonctionnalités nécessaires à sa résolution, un LLM peut le déduire à partir de sa connaissance générale et donc donner des suggestions beaucoup plus intéressantes qu'une simple recherche par similarité des embeddings.

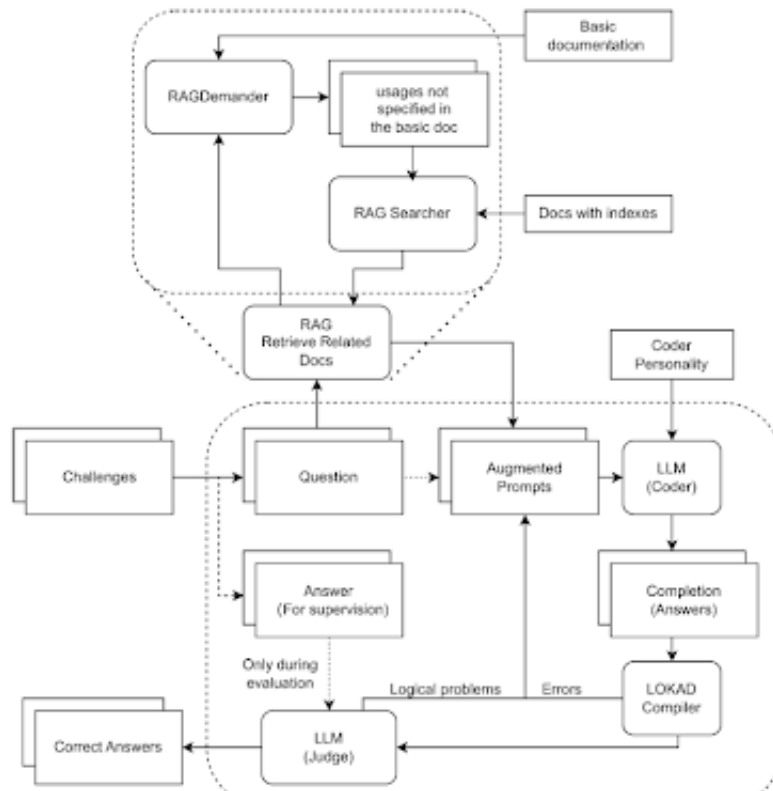


FIGURE 4 – Pipeline de programmation, avec RAG-demander

En pratique, nous avons obtenu cette fois une vraie amélioration : nous sommes passés à un score un peu plus élevé, de 11 challenges compilables dont 10 acceptés par le juge, sur le benchmark de 15 questions. En particulier, certaines fonctions sont bien retrouvées par cette méthode. Un exemple est la fonction `argmax`, utilisée pour résoudre le challenge suivant :

```
1 'Define a table T with 5 names with corresponding score. Show the maximum of
   these 5 scores at the tile a1b2, together with the name that achieves this
   best score at c1d2'
```

Elle est correctement proposée par le RAG-demander à partir de sa connaissance, ce qui n'aurait pas été possible si nous avions comparé directement le texte de la question (qui mentionne nulle part le terme `argmax` ou même `max`) avec celui de la documentation de `argmax`.

À ce stade, nous avons donc obtenu une idée générale du fonctionnement et des pistes d'amélioration de la RAG. Notre implémentation rudimentaire 'from scratch' étant cependant moins bien organisée et peu optimisée, nous avons pris la décision de passer à une implémentation plus robuste et plus développée, avec l'outil LangChain.

IV.3 IMPLÉMENTATION AVEC LANGCHAIN

IV.3.1 • UN PLUS GRAND ENSEMBLE DE BENCHMARKS

La base de challenges sur laquelle nous nous basions jusqu'à maintenant pour tester notre modèle étant assez limitée et conçue à la main par nos soins, nous avons décidé après concertation qu'il était nécessaire de trouver une base de benchmarks plus grande et plus exhaustive pour obtenir des résultats plus réalistes et moins biaisés sur nos performances. Lokad disposait d'une base de benchmarks en format .nvn, que nous avons migrée en .json sur la base de nos précédents challenges, afin de n'avoir aucune modification de code de nos pipelines à effectuer pour lancer les expériences sur le benchmark. C'est donc avec une base de 73 challenges dont 16 écrits par nos soins et 57 pré-élaborés par l'entreprise pour des tests antérieurs que nous avons continué de travailler sur notre LLM.

Les fonctionnalités testées par les 57 nouveaux challenges sont réparties de la manière suivante :

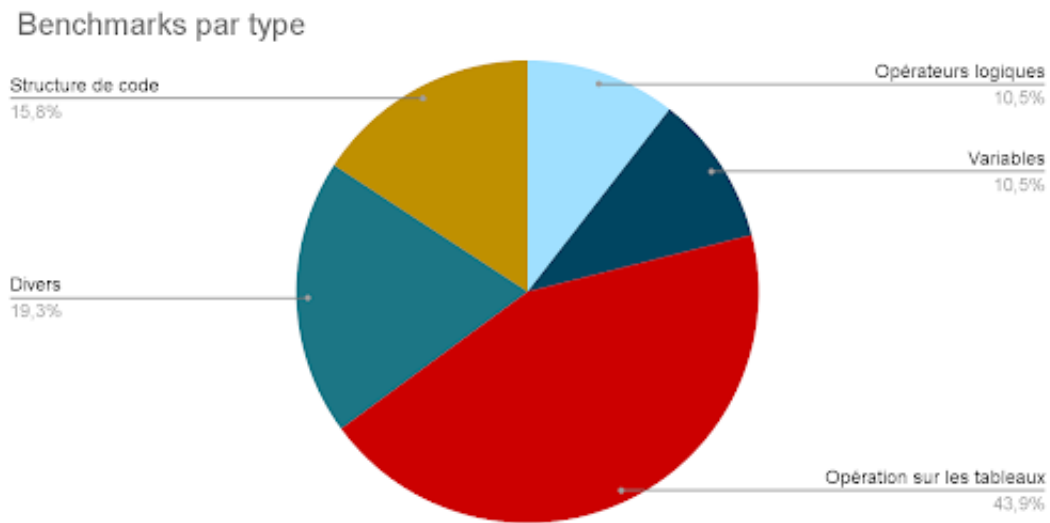


FIGURE 5 – Répartition du contenu des nouveaux challenges du benchmarks

IV.3.2 • INTRODUCTION DE LANGGRAPH ET DE FAISS

LangGraph

LangGraph est un framework conçu pour la construction d'applications à base de LLM dans des contextes multi-agents collaboratifs. Son objectif est de modulariser les modèles de langage en les structurant sous forme de nœuds interconnectés. Ce cadre permet de gérer des tâches impliquant plusieurs étapes, telles que la génération de code, les systèmes de question-réponse, les dialogues multi-tours et l'appel d'outils externes, en construisant des flux de travail intégrés et intelligents.

LangGraph propose un système de modèles de prompt flexible, permettant la composition dynamique d'instructions à partir de variables ou de connaissances externes. Il repose sur une architecture de workflow sous forme de graphes, où chaque nœud peut être un modèle de langage, une fonction, ou même un sous-graphe. Cette structure autorise la gestion de branches, de conditions, de cycles, ce qui permet la coordination efficace de plusieurs agents.

La gestion du contexte est explicite, avec des mécanismes pour transmettre les états intermédiaires, maintenir des variables globales ou persister la mémoire conversationnelle. Elle est essentielle pour la coordination entre agents ou pour faire évoluer les comportements en fonction de l'historique. Le système de typage garantit une

déclaration rigoureuse des entrées et sorties de chaque composant, avec un support natif pour les modèles de données Pydantic.

LangGraph s'intègre aux bases de données vectorielles comme FAISS, Chroma ou Weaviate, permettant des structures de génération augmentée par la récupération (RAG). Le code est complété à partir de connaissances indexées.

FAISS

FAISS (Facebook AI Similarity Search) est une bibliothèque développée par Meta AI pour effectuer des recherches de similarité à grande échelle sur des vecteurs de haute dimension [21]. Avec FAISS, nous pouvons indexer efficacement des milliers à des millions d'embeddings, puis de retrouver rapidement les vecteurs les plus proches d'une requête donnée selon diverses métriques (cosinus, L2, etc.).

Dans notre projet, FAISS est utilisé comme moteur de recherche sémantique pour le système RAG. Concrètement, nous générons des embeddings pour les morceaux de documentation Envision (grammaire, fonctions de référence, exemples) à l'aide de Sentence-Transformers, puis les indexons dans une base FAISS. Lorsqu'une question est posée, son embedding est comparé à cette base afin de récupérer les extraits les plus pertinents à inclure dans le prompt du LLM.

L'avantage principal de FAISS dans notre contexte est sa flexibilité (choix de structures d'indices, support de recherches approximatives via IVF, HNSW, etc.) Dans notre implémentation, nous avons utilisé l'indexation plate (HNSW). Bien que d'autres structures comme IVF ou PQ puissent être explorées pour un passage à l'échelle, cette configuration s'est avérée suffisante dans le cadre expérimental du PSC.

IV.3.3 • NOTRE IMPLÉMENTATION

L'implémentation modulaire concrète et bien structurée qui combine des LLM avec un langage spécifique au domaine DSL. Le projet est organisé en plusieurs modules :

Nous avons développé une architecture modulaire, structurée en plusieurs modules distincts, comme illustré dans la figure suivante :

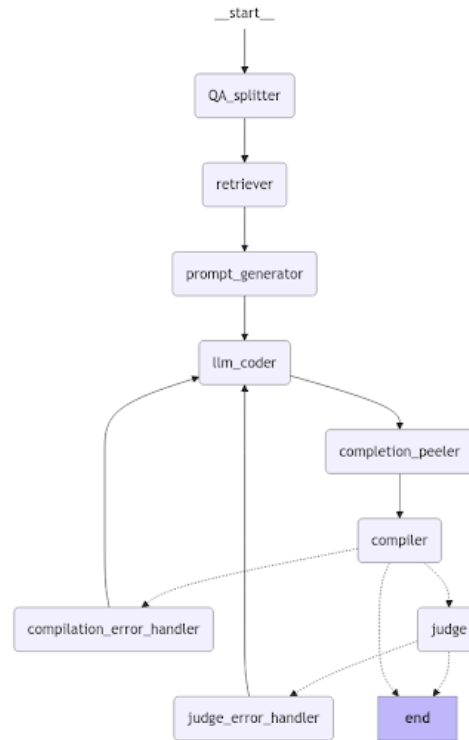


FIGURE 6 – La structure de notre implémentation avec LangChain

Module agents : Ce module centralise les interactions avec les modèles de langage (LLM). Il contient un agent codeur, un agent juge, ainsi qu'un agent *retriever* utilisé pour la RAG

Module RAG : Ce deuxième module contient les composants fondamentaux liés à la RAG et à la gestion des états pour le système de récupération et de génération.

Nous utilisons la base de données vectorielle FAISS pour effectuer les opérations de recherche sémantique. Nous avons apporté un soin particulier au nettoyage et à la structuration des données, afin de garantir la qualité des documents indexés et la pertinence des résultats.

Module Benchmarks : Il regroupe les ensembles de données de documentation Envision et le jeu de test, et donc à évaluer les performances du modèle

Flux de travail

Construction du prompt : Les prompts sont construits dynamiquement en fonction de paramètres (type de tâche, références, personnalité du modèle) définis dans les fichiers de configuration.

Le module d'évaluation de la réponse : Après avoir reçu un prompt structuré, ce module appelle le LLM via un client configuré, il analyse la réponse pour déterminer si elle est correcte ou incorrecte, d'abord par le compilateur et ensuite par le juge, il met enfin à jour l'état global avec les résultats du jugement.

Gestion des erreurs : En cas d'erreur (par exemple, une réponse inattendue du modèle), la fonction capture l'exception et met à jour l'état global avec un message d'erreur, pour permettre éventuellement une nouvelle tentative.

Expérimentations et visualisation : Le répertoire ‘experiments’ contient des scripts et des notebooks voués à tester et visualiser les résultats. Pour nous concentrer sur le jeu de données plutôt que la programmation, nous avons implémenté un outil d’analyse de données afin de déterminer rapidement les écarts entre la réponse donnée par le LLM et la correction (ground truth).

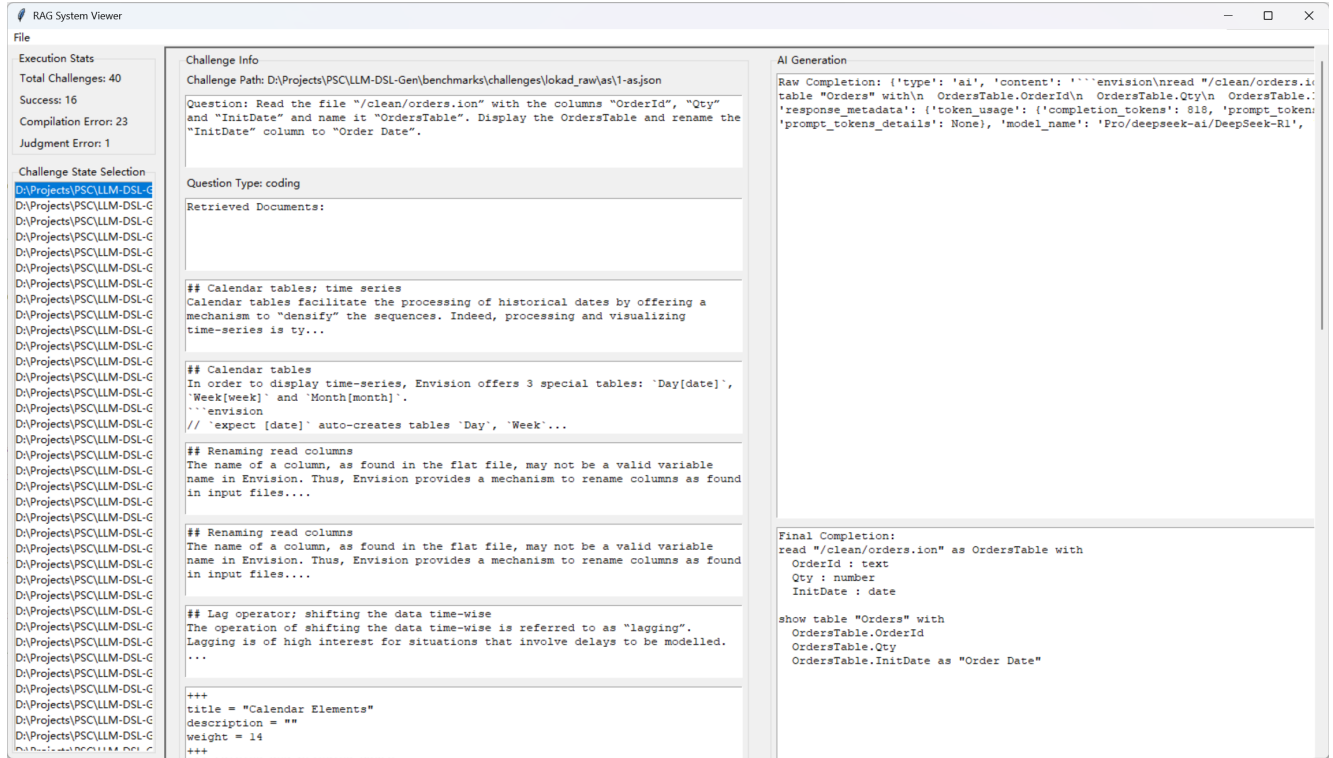


FIGURE 7 – Interface d’affichage des résultats

IV.3.4 • LES DIFFÉRENTS RÉSULTATS

Structure basique

Dans notre configuration de référence, nous fixons le nombre de documents récupérés par RAG à 10, et autorisons jusqu’à trois tentatives par question. En cas d’échec, le message d’erreur est renvoyé au modèle, comme un système de rétro-correction. Nous utilisons le LLM GPT-3.5, sélectionné pour son compromis de coût et de performance.

Résultat : Les codes fournis ont obtenu un score moyen de 12 sur 16 (dans le petit benchmark) et 41 sur 73 dans le benchmark étendu. Ces performances dépassent celles de l’ancien modèle from scratch, notamment grâce à l’efficacité du score de similarité plus optimisé que procure FAISS.

Chaîne de pensée (*chain of thought*)

Lorsque DeepSeek et OpenAI ont publié leurs modèles respectifs R1 et o1, qui peuvent afficher leur raisonnement avant de répondre définitivement à la question (avec l’option "chaîne de pensée"), nous avons réfléchi à utiliser le potentiel de cette nouvelle fonctionnalité.

Nous avons alors essayé de remplacer le modèle GPT-3.5 par DeepSeek R1, en gardant les autres paramètres d’expérience constants, puis nous avons fait quelques expériences sur l’utilisation de la chaîne de pensée.

Résultat : R1 a pu obtenir un score de 13 à 15 sur notre petit benchmark de 16 questions (selon l'aléa), mais sur le gros benchmark sa performance reste moins élevée que GPT-3.5, avec moins de 40 réponses correctes.

Pour interpréter ce résultat, nous avons supposé que ce qui était un obstacle à la performance de notre modèle à ce stade n'est pas la capacité de raisonnement logique du modèle, mais plutôt l'accès aux informations nécessaires à la résolution des challenges, qui empêche l'utilisation correcte des grammaires et des fonctions. Utiliser DeepSeek R1 n'apporte donc pas de réelle plus value. C'est pourquoi nous avons décidé de conserver le LLM GPT-3.5 pour la suite des expérimentations.

Ajout de la documentation résumée (Envision brief)

Dans l'implémentation avec LangChain, nous avons initialement choisi de donner au LLM codeur seulement l'accès aux morceaux extraits par RAG, et pas à la documentation résumée (Envision brief). Nous avons alors essayé d'ajouter le texte d'Envision brief au prompt du codeur, dans l'espoir d'obtenir une amélioration des résultats.

Résultat : L'expérience nous a montré que les performances se sont détériorées en ajoutant Envision brief. En effet, le fait qu'il y ait beaucoup d'informations peu concises dans le fichier envision-brief brouille la compréhension du LLM codeur, comme nous l'avions observé avec les expériences de Prompt Engineering. C'est un résultat à première vue décevant, mais cela montre bien que les morceaux de documentation extraits par RAG sont strictement suffisants pour que le LLM codeur réponde au challenge, donc que notre méthode fonctionne bien !

Augmentation de la longueur de contexte

Pour tester la capacité 'maximale' du RAG, nous avons augmenté le nombre de morceaux à extraire par RAG de 10 à 20. Dans les cas où la longueur du prompt obtenu avec 20 morceaux dépasse la limite imposée par OpenAI, nous retirons simplement les derniers morceaux de manière à respecter la limite. Ce changement de paramètre permettrait d'optimiser les chances de sélectionner les morceaux de documentation pertinents, mais risquait aussi d'ajouter trop de contenu superflu.

Résultat : Le score est passé de 41 à 45. Cette amélioration semble indiquer que l'ajout d'informations supplémentaires, si elles sont bien sélectionnées, ne nuit pas au raisonnement du modèle. La qualité et l'organisation des documents extraits jouent ici un rôle clé.

Réorganisation dynamique des documents

Nous avons enfin testé deux variantes sur la version à 10 documents : réarranger aléatoirement l'ordre des extraits après chaque échec, et remplacer une moitié des documents par les suivants dans le classement de similarité en cas d'échec.

Ces tentatives visaient à atténuer un possible biais de position dans le traitement des prompts par le LLM : un LLM attribue parfois une importance plus importante aux textes placés à différentes positions dans le prompt.

Résultat : Lors de l'expérience, une légère amélioration (1 à 2 points) a parfois été observée sur le benchmark complet. Toutefois, cette variation semble faire partie de l'aléa de génération, donc nous n'avons pas pu considérer ces fonctionnalités comme significativement bénéfiques à notre modèle et nous avons fait le choix de ne pas les conserver.

IV.3.5 • ANALYSE DES RÉSULTATS PAR CATÉGORIE

Intéressons nous enfin à la répartition des résultats selon la capacité algorithmique évaluée. Comme nous l'avons présenté en sous-section IV.3.1, la version étendue du benchmark est structurée, les challenges sont classés par mot clé essentiel à leur résolution. Identifier les mots clés les mieux et moins bien maîtrisés par le LLM codeur permettra d'identifier les limites de notre modèle, ainsi que ses points forts. Le diagramme suivant

a été réalisé à partir des meilleurs résultats obtenus avec l’implémentation FAISS (celui avec 20 morceaux de documentations) :

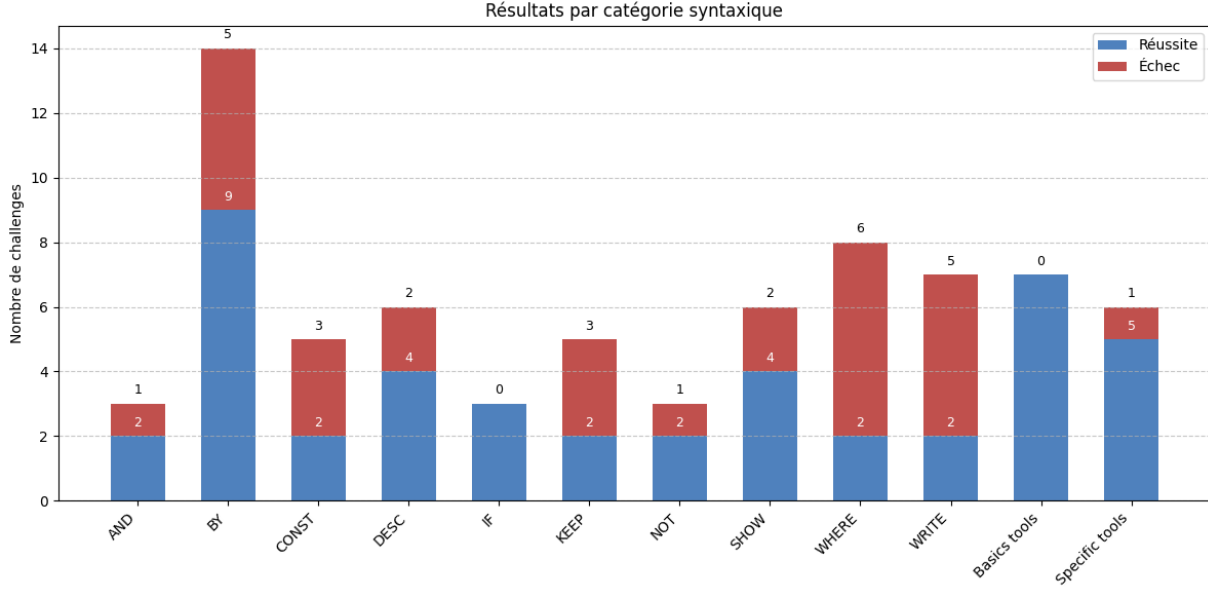


FIGURE 8 – Meilleurs résultats obtenus avec l’implémentation FAISS, classés par catégorie de challenge

Une première observation notable est que notre LLM codeur a un taux de réussite sensiblement plus élevé dans les catégories de mots clés dont la syntaxe diffère peu d’autres langages d’analyse de données, comme le SQL. Par exemple, notre LLM codeur semble avoir bien compris la syntaxe du “IF” en envision, mais a en revanche beaucoup de mal avec les syntaxes utilisant le “WHERE”, car la plupart sont spécifiques au langage envision et ne font vraisemblablement pas partie des données d’entraînement des modèles que nous utilisons. Toutes les stratégies de RAG que nous avons implémentées ont échoué à résoudre les challenges difficiles de la catégorie “WHERE”, mais nous estimons que, conformément à notre hypothèse selon laquelle le LLM utilisé est habitué à d’autres syntaxes que celle du langage envision, une phase de Fine-tuning sur des extraits de code Envision devrait permettre au codeur de surmonter les difficultés précédentes, même si nous n’avons malheureusement pas eu le temps de le tester.

Ensuite, la catégorie Specific tools correspond aux challenges qui nécessitent l’emploi d’une fonction ou d’un type de données assez spécifique, comme afficher un diagramme circulaire ou calculer un argmax. Le seul challenge qui échoue est justement celui avec argmax, car cette fonction a une syntaxe assez particulière. Les autres compétences spécifiques évaluées, comme l’affichage d’un diagramme circulaire, nécessitent pour la plupart seulement la connaissance d’un ou plusieurs mots clés qui s’insèrent dans une syntaxe envision générique. Notre interprétation de ces résultats est que le LLM codeur est capable d’apprendre les syntaxes fondamentales du langage envision, et que la RAG lui permet de trouver efficacement les bons mots clés et fonctions à utiliser. Sa faiblesse provient plutôt de la difficulté de prendre en main les syntaxes complexes et peu répétitives, comme celles d’argmax et celles utilisant le mot clé “WHERE”.

Finalement, notre LLM codeur a une très bonne capacité à réaliser les tâches d’affichage et d’opérations logiques, et peut aussi réaliser les opérations de regroupement et de tri de manière satisfaisante. Il est en revanche plus limité sur l’interaction avec des fichiers et les opérations de filtrage dans une table, un défaut majeur puisque filtrer est une opération cruciale aux langages d’analyse de données. La perspective d’amélioration principale est de fine-tuner le LLM codeur de manière ciblée sur ses points faibles.

V

PISTES D'APPROFONDISSEMENT

V.1 FILTRE DE CONNAISSANCE

Selon l'article suivant [22], il est possible d'améliorer la capacité de sélection de textes pour une méthode de type RAG, si l'on entraîne un filtre (un classificateur) qui détermine si un morceau de texte est lié à la question. Le morceau sera rejeté si la correspondance n'est pas suffisante.

Dans notre cas, nous avons aussi des morceaux de documentation à choisir, mais pas (comme dans l'article) pour répondre aux questions, plutôt pour coder un programme. Cette idée resterait néanmoins intéressante à implémenter car elle pourrait permettre d'améliorer la pertinence des morceaux de texte choisis. Deux difficultés apparaissent tout de même avec cet approche. La première est dans le fait qu'il faut énormément de textes munis de leur classification (pertinent VS non pertinent) pour entraîner un tel classificateur, ce qui n'est pas facile à obtenir dans notre cas. La deuxième est dans le fait que nous ne savons pas si un tel classifieur pourrait être performant. En effet ce système se rapproche de celui du RAG-demander que nous avons construit, qui est assez limité.

V.2 FINE-TUNING

Une autre perspective complémentaire pour améliorer la performance de notre modèle est d'effectuer un Fine-tuning léger du modèle codeur sur un corpus de challenges Envision, concentré sur les lacunes du codeur et très structuré, pour que le LLM puisse mieux assimiler la syntaxe et limiter les hallucinations syntaxiques. Ce travail nécessiterait toutefois une préparation longue et rigoureuse du jeu de données d'entraînement. Nous pouvons mentionner aussi le risque de sur-apprentissage, qui survient lorsque le jeu d'entraînement est trop grand et qui empêcherait le LLM de se détacher des challenges sur lesquels il aurait été entraîné, et donc de perdre en adaptabilité sur des challenges qui ne leur ressemblent pas.

V.3 UN BENCHMARK PLUS ÉQUILIBRÉ

Notre benchmark actuel est déjà varié et de taille raisonnablement grande, mais il présente une concentration sur quelques cas types qui ne couvrent pas l'étendue du langage Envision. Un élargissement vers un ensemble de notions plus varié, couvrant équitablement les différents aspects syntaxiques et fonctionnels d'Envision (agrégats, tableaux, filtrages, affichages conditionnels, etc.), permettrait d'évaluer les performances des modèles de manière plus robuste et plus représentative. Ce besoin d'un benchmark complet est d'ailleurs soutenu par plusieurs travaux récents qui soulignent l'efficacité du Fine-tuning supervisé sur des tâches spécialisées, même avec un volume de données modéré [23] [24]. Dans notre cas, nous avons volontairement gardé ce travail comme une ouverture, car le benchmark dont nous disposons actuellement suffit à l'évaluation de notre modèle RAG.

V.4 ALGORITHMES DE GRAPHE

Une dernière piste que nous avons sérieusement étudiée consiste à s’inspirer des représentations vectorielles de graphes, plus spécifiquement des approches de Graph Embedding appliquées aux LLMs (voir par exemple [25] [26]). L’idée serait de représenter l’ensemble de la documentation sous forme d’un graphe de connaissances au lieu de morceaux de textes, et d’extraire dynamiquement un sous-graphe pertinent pour le fournir en entrée au LLM. Cette méthode est souvent appelée KAG (pour *Knowledge-Augmented Generation*) et consiste en une adaptation de la RAG. Cette approche soulève néanmoins plusieurs difficultés : la documentation est centrée sur des codes informatiques, que l’on ne peut pas traduire en information sous forme de graphe de la même manière que les textes. Dans un graphe de connaissance, la norme est en effet de convertir les textes en triplets canoniques de type **sujet-prédicat-objet**. D’autre part, fournir un sous-graphe en entrée au LLM au lieu d’un texte ne fonctionnerait pas immédiatement. Il faudrait soit effectuer un Fine-tuning du LLM afin de lui apprendre à analyser un graphe de connaissance, soit de manière plus simple traduire le sous-graphe en extrait textuel, mais on y perdrait l’avantage d’avoir structuré les données. Dans le cadre de notre projet, l’utilisation d’algorithmes de graphe de connaissance nous semble donc trop laborieuse : nous n’aurions su en tirer profit dans le temps qui nous était imparti.

VI

GESTION DU PROJET

VI.1 DÉROULÉ ET ORGANISATION DU PROJET

Nous avons structuré l'avancement du PSC ainsi : nous nous sommes réunis de manière hebdomadaire les mercredis après-midi durant les créneaux réservés au PSC. De plus, nous avons planifié des réunions bi-mensuelles (voire hebdomadaires selon les périodes) avec Lokad pour leur faire part de notre avancée et nous mettre au clair sur leurs attendus. Nos réunions hebdomadaires en groupe se sont faites en salles de PC, et les réunions avec Lokad, d'une durée variable entre 15 et 30 minutes ont été faites en visio-conférence. Une partie du travail personnel a été effectué durant les créneaux réservés, mais certaines périodes de développement de notre PSC se sont avérées plus prenantes et nous avons tous ponctuellement complété nos réunions hebdomadaires par du travail en autonomie, en particulier pour ce qui a concerné l'analyse de résultats, la correction de bugs ou la prise en main de nouveaux outils informatiques comme GitHub pour certains d'entre nous.

L'encadrement de notre projet se composait principalement de Joannes Vermorel, CEO de Lokad, avec qui nous avons pu définir des objectifs réalistes et pertinents, et de Tristan Oualid, qui nous a guidé sur les sujets à approfondir, et nous a transmis des ressources de travail précieuses comme les challenges du benchmark étendu. Le PSC s'est déroulé en trois phases distinctes : la première a consisté en la familiarisation avec le langage Envision, nécessaire pour établir un diagnostic sur nos résultats futurs ; la deuxième phase a consisté en la prise en main de l'API OpenAI et des différentes données par l'implémentation de notre premier modèle de Prompt Engineering et de notre modèle d'évaluation ; la troisième, la plus longue, s'est concentrée sur le développement d'un modèle performant fonctionnant avec la méthode RAG. Le tableau suivant illustre le travail réalisé dans le cadre du PSC tout au long de l'année.

Tâche	Sept	Oct	Nov	Déc	Janv	Févr	Mars-Avr
Prise en main du langage Envision	X	X					
Implémentation du modèle Prompt Engineering)		X	X				
Conception du benchmark et du système de juge			X	X		X	
Mise en place du modèle RAG from scratch				X	X		
Incorporation de LangChain et de FAISS						X	X
Expérimentations et évaluation des résultats				X		X	X
Rédaction du rapport							X

TABLE 1 – Tableau d'avancement de notre PSC

La première phase s'est déroulée de septembre à octobre. Ce fut, conformément à nos attentes, une phase durant laquelle nous ne nous sommes pas censurés et nous nous sommes tous documentés sur le langage envision, mais aussi sur les clés de résolution de notre problème ou sur le fonctionnement précis des LLM. Nous souhaitions éviter de nous imposer une ligne directrice trop tôt. Après cette phase de familiarisation avec le sujet, nous avons défini les objectifs présentés dans le tableau prévisionnel en annexe, et commencé notre deuxième phase : la mise en place d'un premier modèle et de son système d'évaluation, d'octobre à décembre. Nous avons alors commencé à diviser les tâches : deux d'entre nous ont travaillé sur la prise en main de l'API OpenAI et la structure générale du dossier GitHub. Deux autres ont travaillé sur l'incorporation du juge dans notre projet, tandis que le dernier

a construit un premier benchmark de challenges Envision et a donc continué à se former sur ce langage. Cette répartition des tâches s'est effectuée naturellement en fonction des goûts et des compétences de chacun. Lorsque nous avons obtenu un code structuré et fonctionnel en décembre, nous avons commencé la troisième phase, qui a duré jusqu'en avril. Nous avons conservé une répartition des tâches similaire à la deuxième phase, avec deux personnes travaillant sur l'architecture générale de notre code, deux autres travaillant sur l'implémentations des différentes fonctionnalités de la RAG, et une personne travaillant sur l'analyse des résultats et l'élargissement du benchmark, même si nous avons à de multiples occasions travaillé sur d'autres tâches que celles liées à notre rôle selon l'avancement de notre PSC.

VI.2 ENSEIGNEMENTS TIRÉS

Nous avons beaucoup appris tout au long de ce projet scientifique collectif et ce, sur différents aspects.

Tout d'abord, le travail collectif a été un vrai enjeu de notre PSC, notamment au début où nous cherchions tous à nous former en Envision de notre côté. Finalement nous avons trouvé que le plus efficace pour nous était d'avancer chacun de notre côté la semaine et de confronter les résultats obtenus lors de nos réunions hebdomadaires. Ainsi nous nous sommes assurés que tout le monde gardait les objectifs communs en tête tout en conservant une autonomie individuelle qui permettait d'être efficace. Chacun a pu progresser à son rythme tout en profitant de l'émulation intellectuelle du groupe et des conseils de ses camarades. Nous avons toujours cherché à maintenir une curiosité dans notre travail, si nous avons exploré des approches aussi variées c'est que nous avons laissé chaque membre du groupe essayer une approche personnelle du problème. L'exploration de ces différentes pistes est, par ailleurs, l'un des éléments clés ayant permis les avancées de notre projet.

Le travail que nous avons mené pendant ce PSC a également été extrêmement instructif car il nous a permis de découvrir un sujet au cœur de l'actualité scientifique. En effet, tous les membres du groupe n'avaient pas forcément d'expérience en manipulation de LLM. Nous avons exploré ce vaste sujet à la fois en découvrant l'état de l'art mais également en manipulant par nous-mêmes. Aujourd'hui l'intelligence artificielle générative semble devenir omniprésente dans nos vies et nous apprécions avoir maintenant une bonne intuition de son fonctionnement. Le suivi de l'actualité scientifique a permis également de choisir sur quel modèle se baser, et quelles innovations tester. Si certaines ont été infructueuses, il n'empêche que chaque tentative différente nous a gratifié d'un apprentissage supplémentaire.

Nous avons grandement gagné en compétences lors de ce PSC. Nous avons exploré plusieurs des algorithmes fondamentaux du RAG et avons appris à les manipuler et à les implémenter en Python. Nous avons ainsi la sensation d'avoir acquis un nouvel outil à notre disposition qui nous sera sûrement utile dans nos futures carrières. Nous avons également développé des compétences utiles dans le cadre professionnel comme l'utilisation du logiciel GitHub.

Finalement, outre les résultats que nous avons obtenus dans le cadre de notre travail, ce projet nous a énormément apporté que ce soit sur le plan des compétences, de la culture scientifique ou même de la gestion de projet.

VI.3 RÉFLEXION A POSTERIORI SUR NOTRE APPROCHE

Nous avons beaucoup aimé travailler sur ce sujet, et sommes satisfaits des résultats finaux obtenus car ils témoignent d'un réel progrès par rapport à nos premières approches. Cependant, a posteriori, nous aurions changé quelques détails dans notre façon d'aborder le sujet.

Premièrement, nous avons parfois eu du mal à comprendre le code des autres, et nous avons tous passé de longues heures à déboguer les contributions des autres à certains fichiers de code. Ceci était majoritairement causé par la différence dans les choix algorithmiques de chacun, et le bagage informatique initial qui varie selon la filière et le lieu d'apprentissage. Notre organisation interne nous a parfois poussé à travailler à plusieurs sur le même fichier, ce qui nous a considérablement ralenti. Avec le recul, nous pensons que nous aurions dû définir notre organisation de manière à ce que chaque personne s'occupe de fonctionnalités différentes et qu'une personne supervise le fonctionnement parallèle des différents fichiers à travers une pipeline.

Ensuite, lors de nos essais avec la méthode RAG, nous avons commencé par une implémentation "from scratch" avant de s'appuyer sur la bibliothèque FAISS, beaucoup plus optimisée. Cette approche nous a certes permis de comprendre les subtilités du RAG avec beaucoup de recul, mais elle nous a été coûteuse en temps et nous a certainement limité dans les résultats finaux que nous avons obtenus. Nous avons aussi parfois manqué de rigueur sur le suivi des résultats que nous avons obtenus, et il a parfois fallu reprendre les anciens codes pour recalculer les anciens résultats, en particulier ceux de nos premières expérimentations.

CONCLUSION

À l'issue de ce projet, nous estimons avoir obtenu des résultats satisfaisants dans les approches que nous avons étudiées. Cela n'aura pas été aisé, notamment car nos objectifs initiaux, sans être irréalistes, étaient très ambitieux, en particulier en ce qui concerne le Fine-Tuning. Le défi d'adapter un LLM à la tâche de génération de code dans un DSL s'est révélé complexe et riche d'enseignements. Même si nous avons commencé sans grande connaissance sur le sujet, nous nous sommes approprié les méthodes de Prompt Engineering et de RAG, et nous restons convaincus que le choix d'explorer ces deux méthodes en priorité a été à la fois le plus formateur et le plus productif. Si certaines approches n'ont pas atteint les performances espérées – comme les premières itérations de notre RAG from scratch –, elles nous ont permis de progresser et de mieux cerner les conditions dans lesquelles un LLM peut réellement être utile. En faisant preuve de rigueur et d'esprit critique, nous avons exploité ces échecs pour obtenir un modèle final assez performant, capable de résoudre une grande partie des tâches de base du langage Envision.

Même si nous avons la sensation de ne pas avoir exploité pleinement son potentiel car il reste de nombreuses pistes d'approfondissement, nous sommes satisfaits du travail que nous avons accompli. Nous pensons avoir réussi à atteindre partiellement les objectifs que nous avons fixé en septembre, et nous avons eu la sensation d'avoir été libres dans notre approche en cherchant à nous approprier les différentes variantes de la RAG. Avec plus de temps, nous aurions aimé explorer le Fine-Tuning ciblé sur les points faibles du modèle, ou enrichir notre système d'évaluation. Quoi qu'il en soit, ce travail nous a donnés de solides bases pour appliquer ces méthodes à d'autres tâches, d'autres langages, ou d'autres outils d'IA générative.

RÉFÉRENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. doi : 10.48550/ARXIV.1706.03762. URL <https://arxiv.org/abs/1706.03762>.
- [2] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. doi : 10.48550/ARXIV.2001.08361. URL <https://arxiv.org/abs/2001.08361>.
- [3] OpenAI. Gpt-4o system card. *CoRR*, abs/2410.21276, 2024. doi : 10.48550/ARXIV.2410.21276. URL <https://arxiv.org/abs/2410.21276>.
- [4] Yuhui Liu et al. A comparative study on reasoning patterns of openai’s o1 model. *CoRR*, abs/2410.13639, 2024. doi : 10.48550/ARXIV.2410.13639. URL <https://arxiv.org/abs/2410.13639>.
- [5] DeepSeek. Deepseek-r1 : Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025. doi : 10.48550/ARXIV.2501.12948. URL <https://arxiv.org/abs/2501.12948>.
- [6] OpenAI. Openai o3 and o4-mini system card. *OpenAI*, System Card, April 2025, 2025. URL <https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf>.
- [7] Mu Chen et al. Understanding the human-llm dynamic : A literature survey of llm use in programming tasks. *CoRR*, abs/2410.01026, 2024. doi : 10.48550/ARXIV.2410.01026. URL <https://arxiv.org/abs/2410.01026>.
- [8] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. doi : 10.48550/ARXIV.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- [9] HuggingFace and ServiceNow Research. Starcoder : May the source be with you! *CoRR*, abs/2305.06161, 2023. doi : 10.48550/ARXIV.2305.06161. URL <https://arxiv.org/abs/2305.06161>.
- [10] Federico Mora, Justin Wong, Haley Lepe, Sahil Bhatia, Karim Elmaaroufi, George Varghese, Joseph E. Gonzalez, Elizabeth Polgreen, and Sanjit A. Seshia. Synthetic programming elicitation for text-to-code in very low-resource programming and formal languages. *CoRR*, abs/2406.03636, 2024. doi : 10.48550/arXiv.2406.03636. URL <https://arxiv.org/abs/2406.03636>.
- [11] Federico Cassano et al. Multipl-e : A scalable and extensible approach to benchmarking neural code generation. *CoRR*, abs/2208.08227, 2022. doi : 10.48550/arXiv.2208.08227. URL <https://arxiv.org/abs/2208.08227>.
- [12] Jacob Andreas et al. Task-oriented dialogue as dataflow synthesis. *CoRR*, abs/2009.11423, 2020. doi : 10.48550/arXiv.2009.11423. URL <https://arxiv.org/abs/2009.11423>.
- [13] Tao Yu et al. Spider : A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *CoRR*, abs/1809.08887, 2018. doi : 10.48550/arXiv.1809.08887. URL <https://arxiv.org/abs/1809.08887>.
- [14] Tianqi Luo et al. nvbench 2.0 : A benchmark for natural language to visualization under ambiguity. *CoRR*, abs/2503.12880, 2025. doi : 10.48550/arXiv.2503.12880. URL <https://arxiv.org/abs/2503.12880>.

- [15] Y. Zhang et al. Codev : Empowering llms for verilog generation through multi-level instruction tuning. *CoRR*, abs/2407.10424, 2024. doi : 10.48550/arXiv.2407.10424. URL <https://arxiv.org/abs/2407.10424>.
- [16] L. Wang et al. Mpirigen : Mpi code generation through domain-specific language models. *CoRR*, abs/2402.09126, 2024. doi : 10.48550/arXiv.2402.09126. URL <https://arxiv.org/abs/2402.09126>.
- [17] Y.-D. Tsai et al. Efficient llm fine-tuning for code generation with data pruning. *CoRR*, abs/2407.05040, 2024. doi : 10.48550/arXiv.2407.05040. URL <https://arxiv.org/abs/2407.05040>.
- [18] Leo Li, Ye Luo, and Tingyou Pan. Openai-o1 ab testing : Does the o1 model really do good reasoning in math problem solving? *CoRR*, abs/2411.06198, 2024. doi : 10.48550/ARXIV.2411.06198. URL <https://arxiv.org/abs/2411.06198>.
- [19] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt. *CoRR*, abs/2302.11382, 2023. doi : 10.48550/ARXIV.2302.11382. URL <https://arxiv.org/abs/2302.11382>.
- [20] Tianxing Wang et al. A survey on llm-as-a-judge. *CoRR*, abs/2411.15594, 2024. doi : 10.48550/ARXIV.2411.15594. URL <https://arxiv.org/abs/2411.15594>.
- [21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2017. doi : 10.48550/ARXIV.1702.08734. URL <https://arxiv.org/abs/1702.08734>.
- [22] Yukun Huang, Sanxing Chen, Hongyi Cai, and Bhuwan Dhingra. To trust or not to trust ? enhancing large language models' situated faithfulness to external contexts. *arXiv preprint arXiv :2410.14675*, 2024. doi : 10.48550/arXiv.2410.14675. URL <https://arxiv.org/abs/2410.14675>.
- [23] Yujia Zhang, Xiaoyu Wang, Ming Li, and Hao Chen. Coela : Cooperative embodied language agents for multi-agent collaboration. *arXiv preprint arXiv :2402.01680*, 2024. doi : 10.48550/arXiv.2402.01680. URL <https://arxiv.org/abs/2402.01680>.
- [24] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca : An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [25] Bingchen Liu, Yuanyuan Fang, Naixing Xu, Shihao Hou, Xin Li, and Qian Li. Large language models for knowledge graph embedding : A survey. *arXiv preprint arXiv :2501.07766*, 2025. doi : 10.48550/arXiv.2501.07766. URL <https://arxiv.org/abs/2501.07766>.
- [26] Lei Liang, Mengshu Sun, Zhengke Gui, Zhongshu Zhu, Zhouyu Jiang, Ling Zhong, Yuan Qu, Peilong Zhao, Zhongpu Bo, Jin Yang, Huaidong Xiong, Lin Yuan, Jun Xu, Zaoyang Wang, Zhiqiang Zhang, Wen Zhang, Huajun Chen, Wenguang Chen, and Jun Zhou. Kag : Boosting llms in professional domains via knowledge augmented generation. *arXiv preprint arXiv :2409.13731*, 2024. doi : 10.48550/arXiv.2409.13731. URL <https://arxiv.org/abs/2409.13731>.